CS771 Minor Assignment 1

Diya Patel diyahp24@iitk.ac.in
Saurya Singh sauryas24@iitk.ac.in
Medini Mudraboina medinim24@iitk.ac.in
Shruti Sahu shrutisahu24@iitk.ac.in
Vasu Goyal vasug24@iitk.ac.in
Deepak Kumar deepakk24@iitk.ac.in

1 Question 1

In the function genChallengeData, the values μ_1, μ_2 and μ_{neg} have the following expressions:

1. $\mu_1 = [-\sqrt{2} - \frac{1}{\sqrt{2l}}, \sqrt{2}]$
2. $\mu_2 = [\sqrt{2} + \frac{1}{\sqrt{2l}}, -\sqrt{2}]$
3. $\mu_{neg} = [-\sqrt{2} - \frac{1}{\sqrt{2l}}, -\sqrt{2}]$

where l represents the variable $level$
The data generated will be on the circles represented by circles centered at μ_1, μ_2 and μ_{neg} with radius 1. Also, the circles centered at μ_1 and μ_2 constitute the data with the "positive" label and the circle centered at μ_{neg} constitute the data with the "negative" label as shown in the figure below.

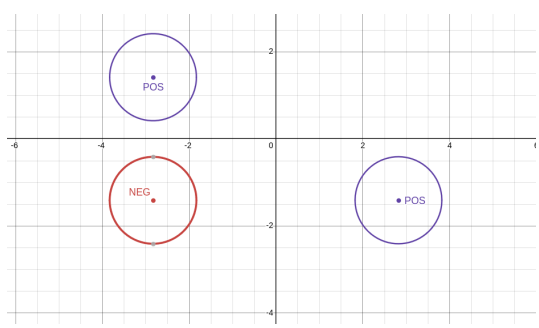


Figure 1: Representation of the generated data

Let $x_0 = \sqrt{2} + \frac{1}{\sqrt{2l}}$ For this data to be linearly seperable, there must exist a line that seperates the "positive" and "negative" value of the data regardless of the value of $level$ and n. Consider the midpoints μ_1, μ_{neg} and μ_2, μ_{neg}, call them ϕ_1 and ϕ_2 respectively

1. $\phi_1 = [-x_0, 0]$
2. $\phi_2 = [0, -\sqrt{2}]$

Consider the line formed by joining ϕ_1 and ϕ_2, call it L

$$L \equiv yx_0 + \sqrt{2}x + \sqrt{2}x_0 = 0$$

Note that the distance from all the centers of the circles to this line is $\frac{\sqrt{2}x_0}{\sqrt{x_0^2+2}}$, and by the way the line is constructed using the midpoints of the positive centers and the negative center, the positive circles

always lie on one side and the negative circle on the other side.
Now we just need to show that the complete surface of each circle lies on one side of the line L
It is given then $l \geq 0$

$$\sqrt{2} + \frac{1}{\sqrt{2}l} \geq \sqrt{2} \implies x_0 \geq \sqrt{2} \implies x_0^2 \geq \sqrt{2} \implies 2x_0^2 \geq x_0^2 + 2$$

$$\implies \sqrt{2}x_0 \geq \sqrt{x_0^2 + 2} \implies \frac{\sqrt{2}x_0}{\sqrt{x_0^2 + 2}} \geq 1$$

Here equality is only achieved when $l \to \infty$. Clearly, the distance of the line L from the center of each circle is more than the radius, hence all the data points on a circle lie on one side of the line, as shown in the figure

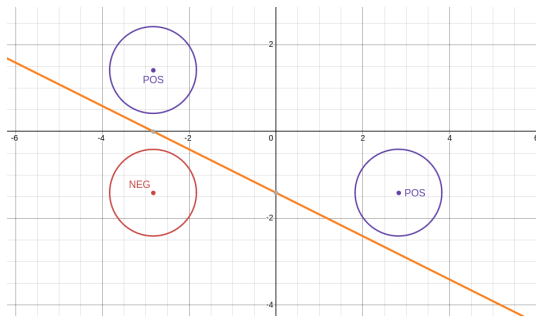


Figure 2: Data with linear seperator

Hence, the data generated by genChallengeData is linearly seperable.

2 Question 2

Table 1: Accuracy for hyperparameter combinations at level = 0.3

myTol	**max_iter**	**myC**			
		0.005	0.010	0.050	0.100
0.1	5	0.913333	0.926667	1.000000	1.000000
	10	0.913333	0.926667	1.000000	1.000000
	100	0.913333	0.926667	1.000000	1.000000
0.01	5	0.913333	0.926667	1.000000	1.000000
	10	0.913333	0.926667	1.000000	1.000000
	100	0.913333	0.926667	1.000000	1.000000
0.001	5	0.913333	0.926667	1.000000	1.000000
	10	0.913333	0.926667	1.000000	1.000000
	100	0.913333	0.926667	1.000000	1.000000
0.0001	5	0.913333	0.926667	1.000000	1.000000
	10	0.913333	0.926667	1.000000	1.000000
	100	0.913333	0.926667	1.000000	1.000000

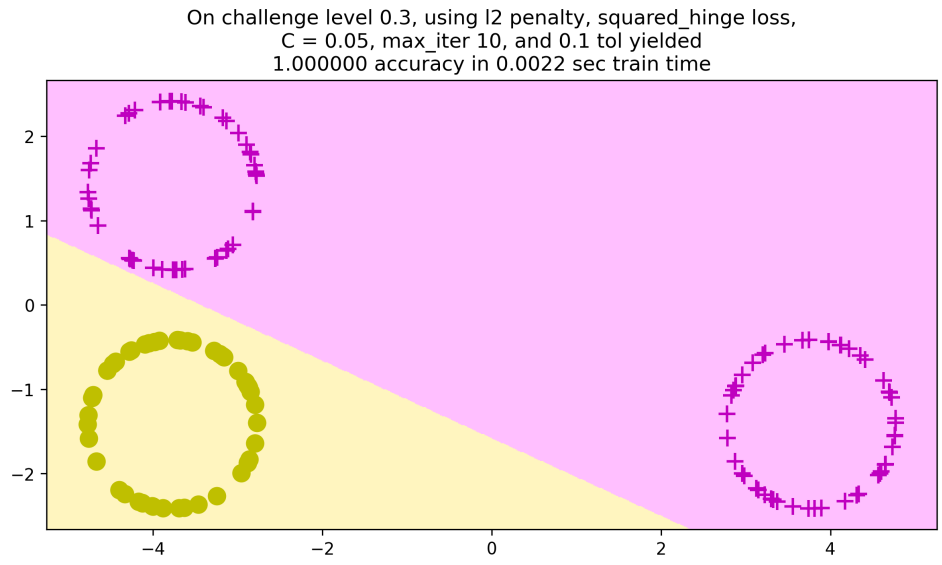
From Table 1, we make the following observations regarding the effect of hyperparameters on classification accuracy at challenge level $= 0.3$:

- **Effect of `tol`:** The tolerance parameter has no effect across all explored values (`tol` $\in \{0.1, 0.01, 0.001, 0.0001\}$). This indicates that for easy data, the solver converges well within the specified tolerance.

- **Effect of** `max_iter`**:** Similarly, the maximum number of iterations has no effect on accuracy across all explored values (`max_iter` $\in \{5, 10, 100\}$). This suggests that the solver converges in small iterations for this challenge level, making the iteration limit a non-binding constraint.
  Note: Configurations with `max_iter` values smaller than 5 consistently triggered `ConvergenceWarning`, indicating that the solver failed to reach the specified tolerance threshold within the allotted iterations. Such configurations were therefore excluded from consideration as they do not guarantee a valid solution, even though they achieved perfect accuracy.

- **Effect of** `C`**:** The regularization parameter `C` is the sole determinant of accuracy. For `C` $\in \{0.005, 0.01\}$, the classifier fails to achieve perfect accuracy, yielding approximately 0.913 and 0.927 respectively. At `C` ≥ 0.05, the classifier achieves 100% accuracy consistently across all values of `tol` and `max_iter`.

Therefore, the optimal hyperparameter configuration yielding 100% accuracy with the smallest `C`, smallest `max_iter`, and largest `tol` is `C` $= 0.05$, `max_iter` $= 5$, and `tol` $= 0.1$.



3 Question 3

From Table 2, we make the following observations regarding the effect of hyperparameters on classification accuracy at challenge level $= 100$:

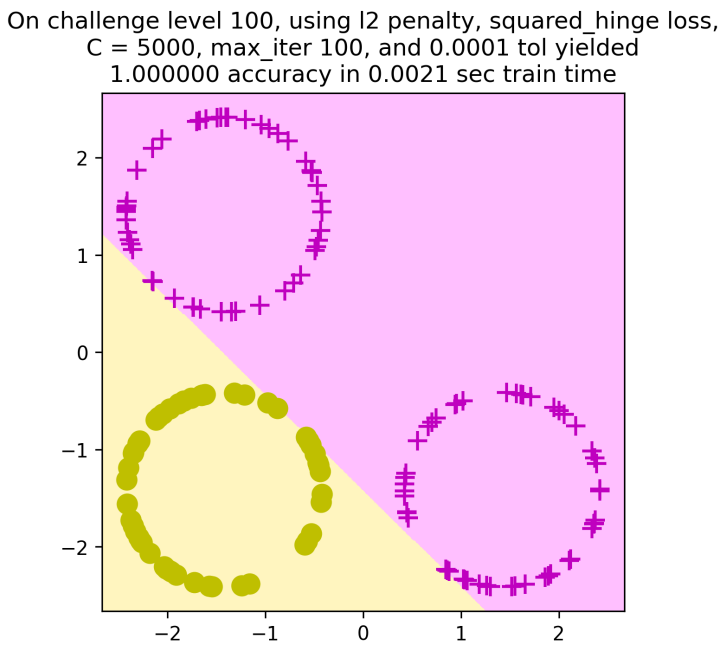
- **Effect of** `max_iter`**:** The maximum number of iterations has no effect on accuracy across all explored values (`max_iter` $\in \{100, 1000, 10000\}$), indicating that convergence behavior at this challenge level is determined entirely by `C` and `tol`.

- **Effect of** `tol` **and** `C`**:** Unlike the low challenge level, accuracy at level $= 100$ is jointly determined by both `C` and `tol`. For `tol` $\in \{0.01, 0.001\}$, the classifier fails to achieve perfect accuracy regardless of `C`, yielding at most 0.993 even for `C` $= 10000$. This indicates that the solver terminates too early to find a sufficiently precise separating hyperplane when the tolerance is large. A smaller value of `tol` ≤ 0.0001 is necessary for the solver to locate the correct decision boundary.

- **Perfect Accuracy:** The transition to 100% accuracy occurs when `C` ≥ 5000 and `tol` ≤ 0.0001 simultaneously. For `C` $\in \{100, 1000\}$, accuracy remains below 1.0 regardless of `tol`, as the regularization is too strong to enforce a correct classification of all training points. This is in contrast to Question 2, where `C` $= 0.05$ was sufficient.

Table 2: Accuracy for hyperparameter combinations at level = 100

tol	**max_iter**	**C**			
		100	1000	5000	10000
0.01	100	0.986667	0.986667	0.986667	0.986667
	1000	0.986667	0.986667	0.986667	0.986667
	10000	0.986667	0.986667	0.986667	0.986667
0.001	100	0.986667	0.986667	0.993333	0.993333
	1000	0.986667	0.986667	0.993333	0.993333
	10000	0.986667	0.986667	0.993333	0.993333
0.0001	100	0.986667	0.993333	1.000000	1.000000
	1000	0.986667	0.993333	1.000000	1.000000
	10000	0.986667	0.993333	1.000000	1.000000
1e-05	100	0.986667	0.993333	1.000000	1.000000
	1000	0.986667	0.993333	1.000000	1.000000
	10000	0.986667	0.993333	1.000000	1.000000



Therefore, the optimal hyperparameter configuration yielding 100% accuracy with the smallest `C`, smallest `max_iter`, and largest `tol` is `C` $= 5000$, `max_iter` $= 100$, and `tol` $= 0.0001$.

4 Question 4

We studied the effect of three hyperparameters: C, tol, and max_iter on the accuracy and training time of LinearSVC at a medium challenge level (myLevel = 5, n = 100, `np.random.seed(42)`). For each hyperparameter, we varied its value while keeping the others value fixed at defaults (C=1.0, tol=$1e-4$, max_iter=1000, penalty='l2', loss='squared_hinge'), so that the effect of each parameter can be studied in isolation.

Effect of C

The parameter C controls the trade-off between maximizing the margin and minimizing training errors.

```
C=   0.001 | Acc=0.8800 | Time=0.00172s
C=   0.010 | Acc=0.9667 | Time=0.00097s
C=   0.100 | Acc=0.9667 | Time=0.00086s
C=   1.000 | Acc=0.9933 | Time=0.00093s
C=  10.000 | Acc=1.0000 | Time=0.00094s
C= 100.000 | Acc=1.0000 | Time=0.00092s
```

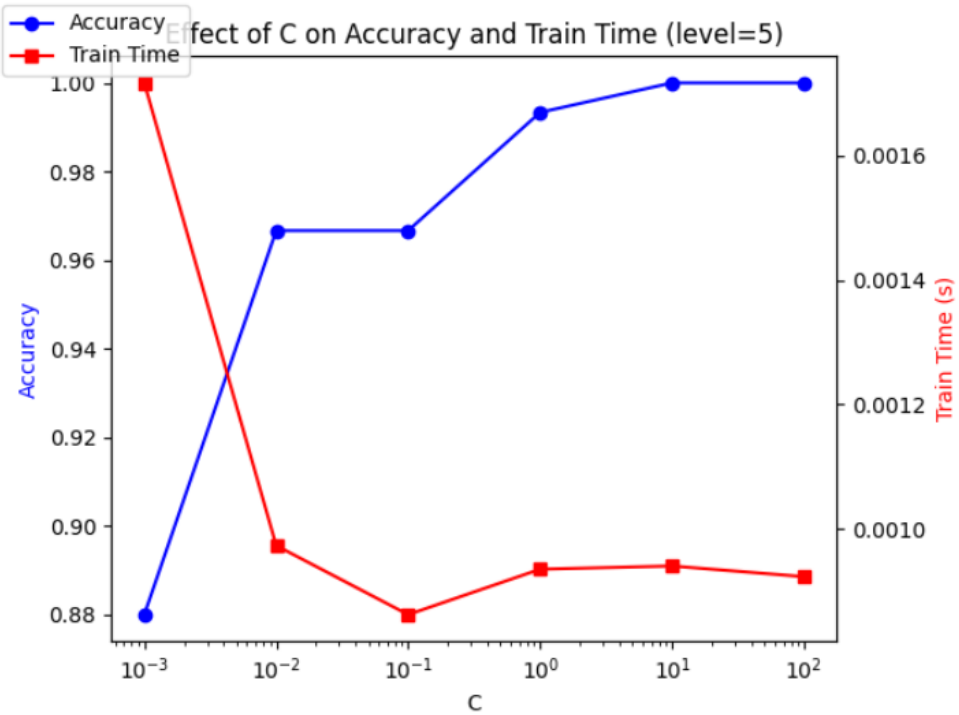


Figure 3: Effect of C on time and accuracy

As seen in Figure 3, accuracy increases with `C`. When `C` is very small (e.g., 0.001), the model is highly relaxed. It prioritizes a smooth, generalized boundary even if it means allowing more misclassifications. This heavy regularization actually makes optimization harder initially, forcing the solver to spend more iterations (and time, peaking at 0.00172s) trying to converge before hitting the `max_iter` limit.

As we turn `C` up to 10 and 100, the model becomes obsessed with the training data, pushing aggressively to classify every single point correctly. The accuracy hits a perfect 100%. Once the model is allowed to fit the data without heavy regularization holding it back, it converges much faster, and the training time drops and stabilizes around 0.0009s.

Effect of tol

The tolerance `tol` determines the stopping criterion for the solver.

We explored `tol` $\in \{10^{-1},\ 10^{-2},\ 10^{-3},\ 10^{-4},\ 10^{-5},\ 10^{-6}\}$.

```
tol=1.0e-01 | Acc=0.9800 | Time=0.00158s
tol=1.0e-02 | Acc=0.9900 | Time=0.00099s
tol=1.0e-03 | Acc=0.9933 | Time=0.00088s
tol=1.0e-04 | Acc=0.9933 | Time=0.00099s
tol=1.0e-05 | Acc=0.9933 | Time=0.00107s
tol=1.0e-06 | Acc=0.9933 | Time=0.00091s
```

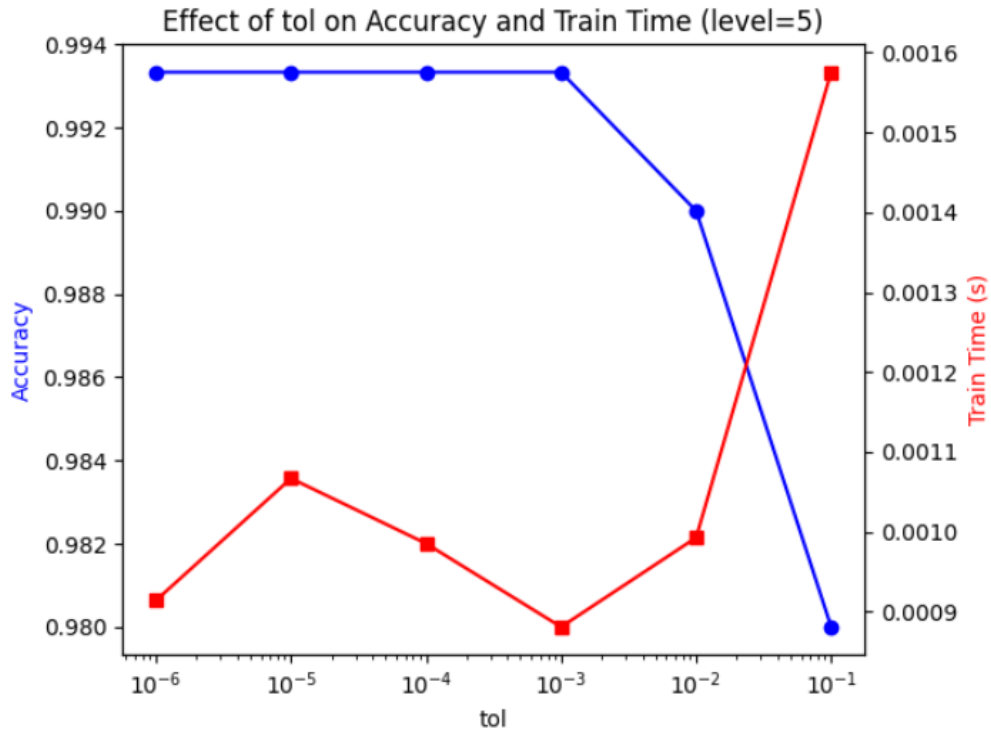


Figure 4: Effect of tol on time and accuracy

As seen in Figure 4, if this tolerance is set too loose (`tol=0.1`), the solver gives up prematurely, before finding a robust decision boundary, resulting in a noticeably lower accuracy of 98%. However, once we enforce a reasonable standard (reducing `tol` below 10^{-3}), the accuracy becomes flat at an optimal 99.33%. At this point, the model has converged to its best possible solution for this dataset and cannot improve further.

Training time shows a slight upward trend for very strict tolerances ($10^{-5}, 10^{-6}$). This shows how the solver requires additional iterations to satisfy a tighter mathematical stopping criterion, though the overall time difference remains minimal for a dataset of this size.

Effect of max_iter

The parameter `max_iter` sets a hard cap on the number of iterations the solver is allowed to run before returning its best attempt.

We explored `max_iter` $\in \{10,\ 50,\ 100,\ 500,\ 1000,\ 5000\}$.

```
max_iter=    10 | Acc=0.9933 | Time=0.00163s
max_iter=    50 | Acc=0.9933 | Time=0.00107s
max_iter=   100 | Acc=0.9933 | Time=0.00092s
max_iter=   500 | Acc=0.9933 | Time=0.00090s
max_iter=  1000 | Acc=0.9933 | Time=0.00098s
max_iter=  5000 | Acc=0.9933 | Time=0.00088s
```

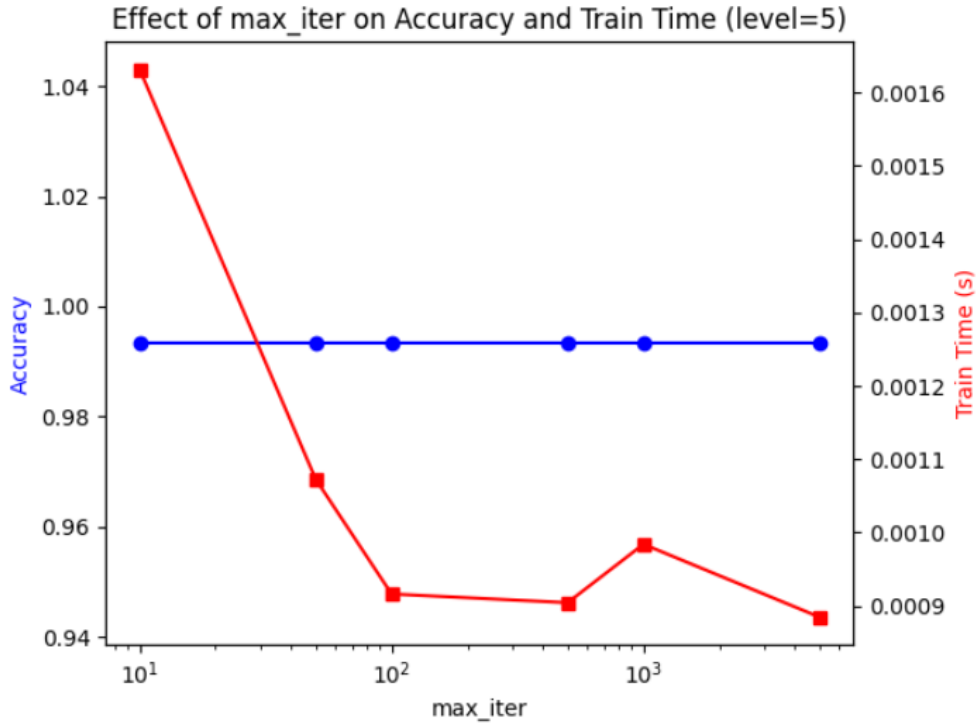


Figure 5: Effect of max_iter on Accuracy and Train Time

Interestingly, accuracy remains perfectly constant at 99.33% across all tested values. This indicates that the solver easily finds the optimal boundary well within the first 10 iterations, granting it a limit of 5000 changes nothing because it finishes early anyway.

The most counter-intuitive finding is the training time. Giving the model the strictest deadline (`max_iter = 10`) resulted in the highest execution time (0.00163s). This happens because, at such a low limit, the `sklearn` library halts the solver prematurely and raises a `ConvergenceWarning`. The computational overhead of generating and printing this text warning actually takes more time than simply allowing the math to finish. Beyond that initial bottleneck, training time stabilizes because the solver consistently converges in the same number of actual iterations, ignoring the artificially high caps.

5 Question 5

5.1 Affect on train time:

As the number of training points n increases, the model takes longer to learn. For smaller datasets (where n is 500 or less), the training time goes up very slowly because it is easy for the computer to process. However, when the dataset gets very large (n = 5000 and above), the time it takes shoots up extremely fast. Our results show this clearly: the average training time jumps as we can see in the table.

5.2 Affect on accuracy:

As the number of training points n increases, we can observe three clear trends: the model's accuracy goes up, the results become much more stable (lower variance).

- **The Initial Drop (n = 10 to n = 100):** When looking at the data, the average accuracy actually seems to drop slightly before it starts increasing. When we use a tiny dataset of just 10 points, the model might get a "lucky" random batch that completely misses the difficult, noisy areas of the problem. Because the data looks too simple, the model can easily achieve a perfect score of 1.00, which artificially inflates the average. However, by the time n reaches 500, the dataset is large enough to consistently include the overlapping points. This forces the model to deal with the noise, bringing the accuracy down to a more realistic starting line.

- **Steady Increase in Accuracy:** After that initial drop, the average accuracy steadily climbs as we add more data. When a model is trained on a very small dataset, it tries too hard to fit every single point. Because there is so little information, one misleading or noisy point carries way too much weight. It acts like a heavy anchor, dragging the model's decision boundary away from where it should actually be, which causes mistakes. But as the dataset grows into the thousands, this problem naturally fixes itself. The huge volume of normal, correct points completely washes out the negative effect of a few bad points. The model can finally see the true overall pattern and safely follow the majority to draw a much more accurate boundary.

- **Results Become Stable (Low Variance):** With small datasets, the accuracy jumps up and down a lot if you run the test multiple times. This happens because every small random batch of data looks slightly different, forcing the model to constantly guess and redraw its boundary in different ways. However, as you provide thousands of data points, the model eventually finds the absolute best decision boundary possible. Once it locks into this optimal position, adding even more data just confirms what it already knows. The boundary stops shifting around, and the accuracy stops fluctuating, settling into a very steady percentage.

Conclusion: This experiment highlights a classic rule in machine learning. Giving the model a massive amount of data is the best way to find a highly accurate and stable decision boundary, but it comes at a cost. As seen in the tables, the training time increases heavily, jumping from a tiny fraction of a second for small datasets to much longer times as n reaches 10,000.

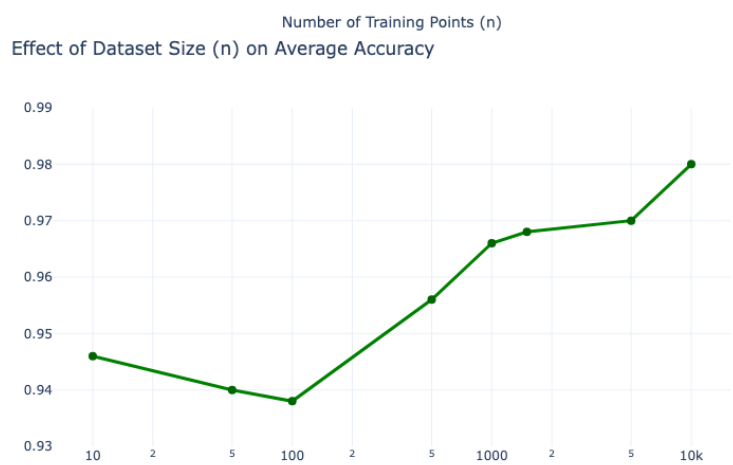


Figure 6: Effect of dataset size (n) on average accuracy.

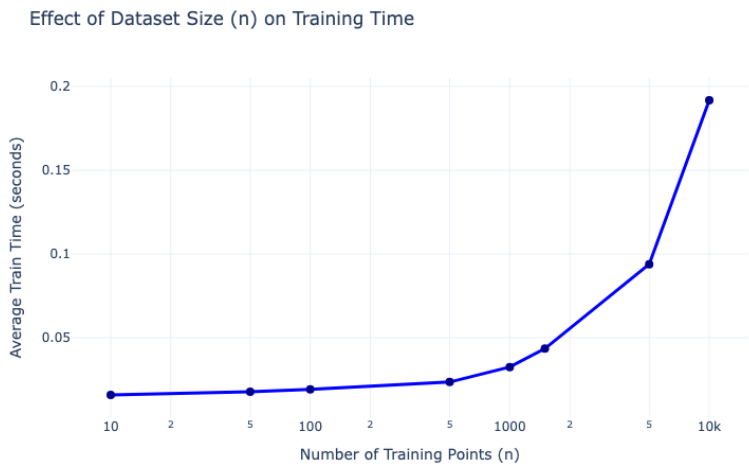


Figure 7: Effect of dataset size (n) on average accuracy.

5.3 Table for Exploring at different values of n on high challenge level (myLevel = 60)

5.3.1 For n = 10:

Accuracy	Train Time(in sec)					Avg. Train Time (in sec)
1.00	0.015	0.018	0.023	0.012	0.016	0.0168
0.93	0.017	0.014	0.011	0.015	0.016	0.0146
0.93	0.018	0.016	0.017	0.017	0.015	0.0166
0.87	0.015	0.014	0.015	0.014	0.019	0.0154
1.00	0.014	0.012	0.022	0.014	0.017	0.0158

Average Results:

$$\text{Average Accuracy} = 0.946$$

$$\text{Average Train Time overall} = 0.01584 \text{ sec}$$

5.3.2 For n = 50:

Accuracy	Train Time(in sec)					Avg. Train Time (in sec)
0.93	0.010	0.014	0.015	0.024	0.017	0.0160
0.93	0.017	0.016	0.015	0.020	0.016	0.0168
0.95	0.029	0.014	0.018	0.020	0.016	0.0194
0.93	0.016	0.018	0.014	0.018	0.020	0.0172
0.96	0.017	0.016	0.026	0.016	0.020	0.0190

Average Results:

$$\text{Average Accuracy} = 0.940$$

$$\text{Average Train Time overall} = 0.01768 \text{ sec}$$

5.3.3 For n = 100:

Accuracy	Train Time(in sec)					Avg. Train Time (in sec)
0.94	0.030	0.017	0.015	0.018	0.017	0.0194
0.95	0.018	0.027	0.019	0.017	0.022	0.0206
0.93	0.020	0.017	0.017	0.029	0.017	0.0200
0.94	0.021	0.022	0.012	0.016	0.021	0.0184
0.93	0.018	0.018	0.020	0.016	0.014	0.0172

Average Results:

$$\text{Average Accuracy} = 0.938$$

$$\text{Average Train Time overall} = 0.01912 \text{ sec}$$

5.3.4 For n = 500:

Accuracy	Train Time(in sec)					Avg. Train Time (in sec)
0.95	0.023	0.021	0.024	0.023	0.026	0.0234
0.96	0.021	0.021	0.023	0.023	0.022	0.0220
0.96	0.028	0.021	0.022	0.024	0.025	0.0240
0.96	0.023	0.032	0.024	0.031	0.024	0.0268
0.95	0.014	0.023	0.023	0.025	0.022	0.0214

Average Results:

$$\text{Average Accuracy} = 0.956$$

$$\text{Average Train Time overall} = 0.02352 \text{ sec}$$

5.3.5 For n = 1000:

Accuracy	Train Time(in sec)					Avg. Train Time (in sec)
0.96	0.024	0.034	0.032	0.034	0.045	0.0338
0.97	0.030	0.024	0.028	0.031	0.033	0.0292
0.96	0.028	0.034	0.027	0.028	0.030	0.0294
0.97	0.032	0.047	0.033	0.034	0.033	0.0358
0.97	0.034	0.034	0.035	0.032	0.037	0.0344

Average Results:

$$\text{Average Accuracy} = 0.966$$
$$\text{Average Train Time overall} = 0.03252 \text{ sec}$$

5.3.6 For n = 1500:

Accuracy	Train Time(in sec)					Avg. Train Time (in sec)
0.96	0.057	0.039	0.035	0.064	0.039	0.0468
0.97	0.051	0.041	0.037	0.052	0.053	0.0468
0.97	0.050	0.037	0.042	0.038	0.047	0.0428
0.97	0.050	0.038	0.038	0.038	0.036	0.0400
0.97	0.038	0.031	0.037	0.040	0.059	0.0410

Average Results:

$$\text{Average Accuracy} = 0.968$$
$$\text{Average Train Time overall} = 0.04348 \text{ sec}$$

5.3.7 For n = 5000:

Accuracy	Train Time(in sec)					Avg. Train Time (in sec)
0.97	0.091	0.089	0.083	0.095	0.092	0.0900
0.97	0.098	0.095	0.087	0.084	0.082	0.0892
0.97	0.125	0.104	0.084	0.102	0.081	0.0992
0.97	0.090	0.084	0.081	0.139	0.131	0.1050
0.97	0.093	0.086	0.083	0.084	0.084	0.0860

Average Results:

$$\text{Average Accuracy} = 0.970$$
$$\text{Average Train Time overall} = 0.09388 \text{ sec}$$

5.3.8 For n = 10000:

Accuracy	Train Time(in sec)					Avg. Train Time (in sec)
0.98	0.178	0.162	0.167	0.179	0.249	0.1870
0.98	0.194	0.174	0.199	0.163	0.224	0.1908
0.98	0.167	0.163	0.220	0.310	0.192	0.2104
0.98	0.177	0.195	0.177	0.170	0.228	0.1894
0.98	0.211	0.178	0.169	0.164	0.184	0.1812

Average Results:

$$\text{Average Accuracy} = 0.980$$
$$\text{Average Train Time overall} = 0.19176 \text{ sec}$$